

SLOT2:

API, Node&Http

Tóm tắt nội dung

API:

4.1 What is API

4.2 How do APIs work?

4.3 What does an api do?

HTTP:

5.1 HTTP Overview

5.2 Receiving an HTTP Request with Express

NodeJS and HTTP:

6.1 Node.JS HTTP Module

6.2 Node.JS as a web server



4. API

4.1 What is API?

- API (Application Programming Interface) là một tập hợp các định nghĩa cho phép phần mềm giao tiếp với nhau.
- Là cầu nối giữa các hệ thống, cho phép chúng truy cập tài nguyên hoặc dịch vụ lẫn nhau.

4.2 How do APIs work?

- Một client gửi **HTTP Request** đến một API endpoint.
- API xử lý và gửi **HTTP Response** lại cho client.
- Quá trình này thường diễn ra qua giao thức **HTTP/HTTPS**.

4.3 What does an API do?

- API định nghĩa cách hai hệ thống tương tác.
- Có thể trả dữ liệu (GET), thêm dữ liệu (POST), cập nhật (PUT), xóa (DELETE), v.v.
- Là xương sống của web services, mobile apps, microservices, v.v.



5. HTTP

5.1 HTTP Overview

- HTTP (Hypertext Transfer Protocol) là giao thức ứng dụng cho client-server.
- Giao tiếp theo kiểu stateless.
- Các phiên bản:
 - HTTP/0.9: lỗi thời
 - HTTP/1.1: phổ biến, mỗi kết nối một request
 - HTTP/2: hỗ trợ nhiều request song song, nhanh hơn

5.2 Receiving an HTTP Request with Express

- Dùng framework **Express** để nhận và xử lý HTTP request:

```
js

const express = require('express');
const app = express();

app.get('/', (req, res) => res.send('Hello World!'));
app.listen(3000);
```

6. NodeJS and HTTP

6.1 Node.JS HTTP Module

- Sử dụng module `http` của Node.js để tạo HTTP server:

```
js

const http = require('http');

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!\n');
});

server.listen(3000);
```

- Có thể kiểm tra với **Postman** hoặc trình duyệt tại `http://localhost:3000`

6.2 Node.JS as a Web Server

- Node.js có thể hoạt động như một **Web Server** độc lập (không cần Apache/Nginx).
- Có thể **serve HTML files** bằng cách đọc file từ thư mục `public`:

```
js

const fs = require('fs');
fs.readFile('./public/index.html', (err, data) => { ... });
```

Bonus:

- Dùng module `fs` để **đọc/ghi file** trong Node.js:

```
js

const fs = require('fs');
fs.readFile('README.md', (err, data) => { ... });

fs.writeFile('README.md', 'Hello World', (err) => { ... });
```

✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

 Bài học 1: API – Slot2.1_API.pptx

1. Khái niệm API

- API là **giao diện lập trình ứng dụng**, giúp **giao tiếp giữa các hệ thống phần mềm**.
- Cho phép tái sử dụng chức năng đã có một cách dễ dàng, tiết kiệm công sức lập trình.

2. Cách hoạt động của API

- Client gửi yêu cầu qua HTTP (GET, POST, PUT, DELETE).
- API xử lý và phản hồi dữ liệu về client (thường là JSON hoặc XML).

3. Phân biệt SOAP vs REST

SOAP	REST
Giao tiếp qua XML	Hỗ trợ nhiều định dạng (chủ yếu JSON)
Cấu trúc chặt chẽ	Linh hoạt, đơn giản hơn
Tính bảo mật cao hơn	Hiệu suất và tốc độ tốt hơn

4. API Gateway

- Là cổng trung gian xử lý các request đến API.
- Thực hiện kiểm tra, xác thực, phân phối request đến microservices, giới hạn tốc độ, ghi log...

5. Các thao tác chính của API

Method	Ý nghĩa
GET	Lấy dữ liệu
POST	Tạo mới dữ liệu
PUT	Cập nhật dữ liệu
DELETE	Xóa dữ liệu

6. JSON vs XML

JSON	XML
Gọn, dễ đọc, dùng phổ biến	Dài dòng, ít dùng hơn
Tốc độ nhanh hơn	Cấu trúc rõ ràng nhưng phức tạp hơn
JavaScript-friendly	Ngôn ngữ đánh dấu mở rộng

 Bài học 2: NodeJS & HTTP – Slot2.2_Node&Http.pptx

1. HTTP – Giao thức nền tảng Web

- Giao thức **stateless**, dùng để truyền dữ liệu giữa client và server.
- Các phương thức (verbs): **GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS**.
- HTTP/2 cải thiện hiệu năng với khả năng **gửi song song nhiều request**.

2. HTTP Request & Response

- Request gồm: method, URL, headers, body.
- Response gồm: status code (200, 404, 500...), headers, body.

3. Node.js HTTP Module

- Dùng `http.createServer()` để tạo server:

js

```
const http = require('http');
http.createServer((req, res) => {
  res.end('Hello World');
}).listen(3000);
```

- Dùng `fs` để đọc/ghi file trả về trình duyệt.

4. POSTMAN

- Công cụ **kiểm thử API**, gửi yêu cầu HTTP và xem phản hồi.
- Hữu ích để học và debug backend server/API.

5. File System trong Node.js

- Dùng module `fs` để:
 - `readFile()`, `writeFile()` – đọc/ghi file
 - `createReadStream()`, `createWriteStream()` – xử lý file lớn
 - `chmod()` – đổi quyền file
- Quan trọng khi xây dựng ứng dụng cần **lưu trữ hoặc phục vụ nội dung tĩnh (HTML, ảnh,...)**.

6. Dựng Web Server cơ bản với Node

- Trả về file `.html` khi truy cập qua đường dẫn tương ứng.
- Dùng module `path` để xác định đường dẫn chính xác.

Ghi nhớ quan trọng:

- **API = Giao tiếp phần mềm** (REST phổ biến hơn SOAP).
- **HTTP = Giao thức truyền dữ liệu nền web.**
- **Node.js = Platform tạo Web Server hiệu quả, không cần phần mềm ngoài.**
- Nắm chắc 4 thao tác HTTP: **GET, POST, PUT, DELETE.**
- **JSON** là định dạng trao đổi dữ liệu **chuẩn hiện nay.**
- Biết cách dùng `http`, `fs`, `path` trong Node.js.

✓ ỨNG DỤNG THỰC TIỄN:

Kỹ thuật	Ứng dụng thực tế
API (REST, JSON)	Kết nối hệ thống, App → Server, thanh toán, tích hợp bên thứ 3
HTTP Methods	Xử lý CRUD: GET (đọc), POST (thêm), PUT (cập nhật), DELETE (xóa)
Node.js HTTP	Xây dựng web server, backend không cần Apache
File System	Tạo log, lưu dữ liệu, hệ thống quản lý tập tin
Stream	Xử lý dữ liệu lớn như log, media, file lớn
Postman	Test API trong quá trình phát triển phần mềm

1. Ứng dụng thực tế của API

◆ Kết nối giữa các hệ thống

- **Ví dụ:** Một app đặt vé máy bay (Traveloka) sử dụng API để kết nối đến API của các hãng bay như Vietnam Airlines, Vietjet...
- **Kỹ thuật sử dụng:** RESTful API, JSON, HTTP GET/POST

◆ Tích hợp thanh toán

- **Ví dụ:** Sử dụng API từ cổng thanh toán (Momo, ZaloPay, PayPal) để thanh toán trực tuyến.
- **Kỹ thuật sử dụng:** API Gateway để xử lý an toàn, xác thực request.

◆ Giao tiếp giữa frontend và backend

- **Ví dụ:** Website bán hàng dùng React ở frontend gửi API request tới Node.js backend để lấy danh sách sản phẩm.
- **Kỹ thuật sử dụng:** REST API, HTTP methods (GET, POST, PUT, DELETE)

◆ Ứng dụng di động và server

- **Ví dụ:** Ứng dụng Grab lấy dữ liệu vị trí tài xế, đơn hàng, trạng thái... từ server thông qua API.
- **Kỹ thuật sử dụng:** API dùng JSON, bảo mật thông qua token, sử dụng POST/GET

◆ Social login và chia sẻ

- **Ví dụ:** Login bằng Google/Facebook, hoặc chia sẻ nội dung lên Twitter từ một website.
- **Kỹ thuật sử dụng:** API của mạng xã hội (OAuth2, REST)

2. Ứng dụng thực tế của HTTP & Node.js

◆ Tạo Web Server đơn giản

- **Ví dụ:** Tạo website cá nhân hoặc portfolio hiển thị thông tin, HTML file.
- **Kỹ thuật sử dụng:** Node.js `http` module, `fs` để đọc file HTML, status code HTTP.

◆ Tạo RESTful API Backend

- **Ví dụ:** Backend cho ứng dụng To-do List, Chat App, quản lý sản phẩm.
- **Kỹ thuật sử dụng:** Node.js với Express, HTTP methods, JSON, routes.

◆ Xây dựng hệ thống File Server

- **Ví dụ:** Ứng dụng upload/download tài liệu cho trường học hoặc nội bộ công ty.
- **Kỹ thuật sử dụng:** Node.js `fs`, `read/write/streaming files`

◆ Thao tác với File Hệ thống

- **Ví dụ:** Tạo ứng dụng CLI để đọc log, lưu dữ liệu log người dùng.
- **Kỹ thuật sử dụng:** Node.js `fs`, `fs.readFile`, `fs.writeFile`, `fs.createReadStream`

◆ Phân tích & xử lý dữ liệu lớn

- **Ví dụ:** Đọc file log dung lượng lớn bằng stream để phân tích từ khóa, số lượt truy cập,...
- **Kỹ thuật sử dụng:** Read/Write Stream – tiết kiệm bộ nhớ, hiệu quả với dữ liệu lớn

◆ Sử dụng Postman để kiểm thử

- **Ví dụ:** Kiểm thử API nội bộ công ty hoặc khi xây dựng dịch vụ backend.
- **Kỹ thuật sử dụng:** Gửi request GET/POST bằng Postman để xem phản hồi từ server

CHI TIẾT MỘT SỐ NỘI DUNG:



◆ 1. API là gì?

- **API (Application Programming Interface)** là giao diện lập trình ứng dụng.
- Cho phép **hai ứng dụng phần mềm giao tiếp với nhau**.
- **Ví dụ:** Ứng dụng điện thoại đặt xe (client) gửi yêu cầu đến server (backend), server phản hồi dữ liệu về thông qua API.



Đặc điểm:

- Cho phép **tái sử dụng các chức năng có sẵn**.
- Trừu tượng hóa logic bên trong hệ thống, **chỉ cần biết cách gọi**.
- Giao tiếp thường dựa trên **HTTP + JSON/XML**.

◆ 2. API hoạt động thế nào?

- Client gửi **HTTP Request** đến server qua API.

- Server xử lý và trả về **HTTP Response**.
- Request gồm: method (GET, POST,...), URL, headers, body (nếu có).
- Response gồm: status code, headers, body (dữ liệu trả về).

◆ 3. Các loại API phổ biến

🌐 a. REST API

- REST = Representational State Transfer
- Giao tiếp dựa trên HTTP và thường dùng JSON.
- Ưu điểm:
 - Đơn giản, hiệu quả
 - Dễ tích hợp với frontend
 - Phù hợp cho mobile, web

📄 b. SOAP API

- SOAP = Simple Object Access Protocol
- Giao tiếp bằng XML, phức tạp hơn REST.
- Dùng nhiều trong hệ thống lớn, có yêu cầu bảo mật cao (ví dụ: ngân hàng).
- Request dùng HTTP POST, nội dung gửi trong XML (SOAP Envelope).

◆ 4. API Gateway là gì?

- API Gateway là **cổng vào duy nhất** cho các API của hệ thống backend.
- Chức năng:
 - Kiểm tra quyền truy cập
 - Giới hạn tần suất request
 - Xác thực người dùng (OAuth, token...)
 - Ghi log, theo dõi lỗi
 - Chuyển đổi giao thức nếu cần (HTTP ↔ gRPC...)

◆ 5. API làm được gì?

- **Giao tiếp giữa client và server**
- Thao tác dữ liệu:
 - **GET** – Lấy dữ liệu
 - **POST** – Gửi dữ liệu mới
 - **PUT** – Cập nhật toàn bộ
 - **DELETE** – Xóa dữ liệu
- Cho phép tích hợp bên ngoài (ví dụ: bản đồ, social login...)

◆ 6. JSON vs XML

Tiêu chí	JSON	XML
Dạng dữ liệu	Văn bản theo cú pháp JS	Văn bản có thẻ đánh dấu
Dễ đọc	Dễ hơn, ngắn gọn	Dài dòng hơn

Hiệu suất	Nhanh, nhẹ	Chậm hơn
Dùng phổ biến	Web/Mobile hiện đại	Hệ thống cũ, SOAP

7. API vs SDK

API	SDK
Giao diện kết nối	Gói công cụ phát triển
Chỉ cung cấp cách giao tiếp	Bao gồm thư viện, công cụ
Ví dụ: REST API	Ví dụ: Android SDK

NodeJS & HTTP

1. HTTP là gì?

- HTTP = HyperText Transfer Protocol
- Là **giao thức truyền dữ liệu giữa Client và Server**.
- **Stateless**: mỗi request độc lập.
- Dựa trên nền TCP/IP.

Các phiên bản HTTP:

- HTTP/0.9: rất cũ, chỉ hỗ trợ GET
- HTTP/1.1: phổ biến nhất, nhưng chỉ một request mỗi kết nối
- **HTTP/2**: hỗ trợ gửi nhiều request song song (multiplexing)

2. Cấu trúc HTTP Request & Response

Request:

- Method: GET, POST, PUT, DELETE,...
- URL: đường dẫn tài nguyên
- Headers: chứa thông tin (token, content-type,...)
- Body: dữ liệu gửi lên (thường với POST/PUT)

Response:

- Status code: 200 OK, 404 Not Found, 500 Internal Server Error...
- Headers: thông tin bổ sung (content-type,...)
- Body: nội dung trả về (HTML, JSON,...)

3. Các phương thức HTTP phổ biến

Method	Mô tả
GET	Lấy dữ liệu
POST	Gửi dữ liệu mới
PUT	Cập nhật toàn bộ dữ liệu
DELETE	Xóa dữ liệu
PATCH	Cập nhật từng phần
HEAD	Lấy thông tin metadata
OPTIONS	Kiểm tra các tùy chọn hỗ trợ

◆ 4. Node.js HTTP Module

✓ Tạo Web Server đơn giản:

```
js

const http = require('http');
http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!\n');
}).listen(3000);
```

- Gõ `http://localhost:3000` để xem kết quả.

◆ 5. Kiểm thử API bằng Postman

- Gửi request giả lập từ client đến server.
- Hữu ích để kiểm tra API: header, body, JSON, code phản hồi.

◆ 6. Làm việc với File System trong Node.js

📄 Các thao tác với file:

```
js

const fs = require('fs');

// Đọc file
fs.readFile('README.md', (err, data) => {
  console.log(data.toString());
});

// Ghi file
fs.writeFile('README.md', 'Hello World', err => {
  if (!err) console.log('Ghi file thành công');
});
```

🔄 Synchronous vs Asynchronous:

- `readFileSync, writeFileSync`: đồng bộ → chậm, block.
- `readFile, writeFile`: bất đồng bộ → tốt hơn.

◆ 7. Stream – Xử lý file lớn

- Dùng cho dữ liệu lớn, không đọc hết vào RAM một lần.

```
js

const fs = require('fs');
const stream = fs.createReadStream('bigfile.txt');
stream.on('data', chunk => {
  console.log(chunk.toString());
});
```

});

8. Xây dựng File Server cơ bản

- Dùng `fs`, `http`, `path` để:
 - Tạo/xóa/đọc/ghi file
 - Trả về file HTML khi truy cập từ trình duyệt
- Kết hợp với module `path` để xử lý đường dẫn chính xác.

9. Tương tác đường dẫn tập tin

- `__dirname`, `__filename`: lấy đường dẫn hiện tại
- `path.resolve`, `path.join`: kết nối đường dẫn
- `path.extname`: kiểm tra phần mở rộng

✓ TỔNG KẾT KIẾN THỨC CỐT LÕI

Chủ đề	Ý nghĩa thực tế
API (REST, SOAP)	Giao tiếp giữa ứng dụng
HTTP	Giao thức truyền dữ liệu client-server
JSON/XML	Định dạng dữ liệu truyền/nhận
Node.js http module	Tạo web server không cần phần mềm ngoài
fs, path	Tương tác hệ thống file: đọc, ghi, stream
Postman	Công cụ kiểm thử API rất quan trọng

KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

✓ 1. KINH NGHIỆM THỰC TẾ

◆ API

🧠 Kinh nghiệm:

- Thiết kế rõ ràng API ngay từ đầu (RESTful, có version: `/api/v1/users`).
- Tách biệt frontend/backend qua API để dễ bảo trì.
- Dùng **status code** hợp lý để client dễ xử lý logic:
 - 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Internal Error...

🧠 Cần trọng:

- Không nên lộ thông tin nhạy cảm qua URL (GET `/users?password=123` ❌).
- Luôn kiểm tra dữ liệu input từ người dùng (validation).
- Dữ liệu trả về nên có cấu trúc nhất quán (dùng JSON schema nếu cần).

◆ HTTP

🧠 Kinh nghiệm:

- Sử dụng đúng method cho từng mục đích:
 - GET cho đọc, POST cho tạo mới, PUT cho cập nhật, DELETE để xóa.
- Phải hiểu **stateless**: mỗi HTTP request không nhớ dữ liệu từ request trước → cần dùng JWT/token nếu muốn giữ trạng thái người dùng.

🧠 Cần trọng:

- **CORS (Cross-Origin Resource Sharing)**: nhiều API bị lỗi do chưa bật CORS.
 - Mẹo: Cài cors trong Express:

```
js
const cors = require('cors');
app.use(cors());
```

◆ Node.js – HTTP & File System

🧠 Kinh nghiệm:

- Sử dụng `express` để rút gọn code thay vì thuần `http`.
- Khi phục vụ file lớn → dùng **Stream**, không dùng `readFile` (tốn RAM).
- Dùng `path.resolve()` để đảm bảo đường dẫn file chính xác và an toàn (tránh lỗi file not found hoặc path traversal attack).

🧠 Cần trọng:

- Nếu bạn đọc file đồng bộ (`fs.readFileSync`) trong server, điều đó sẽ **chặn (block)** toàn bộ request khác → không nên dùng trong sản phẩm thật.
- Ghi log lỗi cẩn thận, tránh lộ thông tin server cho người dùng cuối.

💡 2. TIPS (MẸO HAY)

📌 TIPS CHO API:

- Dùng [Postman](#) để kiểm thử API nhanh, tạo collection test.
- Tạo một file JS chứa URL base, headers, để dễ tái sử dụng (đặc biệt với token auth).
- Có thể dùng **Swagger** hoặc **Postman Docs** để tự động tạo tài liệu API.

📌 TIPS CHO HTTP:

- Khi debug lỗi HTTP, bật console log headers cả ở client và server:

```
js
console.log(req.headers);
```

- Status code dưới 100 hoặc > 599 là không hợp lệ.

TIPS CHO Node.js:

- Dùng `nodemon` để tự động reload server khi thay đổi code:

```
bash

npm install -g nodemon
nodemon index.js
```

- Dùng `fs.promises` (async/await) để viết code gọn hơn:

```
js

const data = await fs.promises.readFile('file.txt', 'utf8');
```

TIPS File System:

- Với upload file lớn (video, ảnh...), dùng `stream` + `multer` hoặc `busboy`.
- Ghi log bằng `fs.createWriteStream()` để theo dõi request/error.

3. CÁC VẤN ĐỀ THƯỜNG GẶP

Vấn đề	Giải pháp
Request API trả về lỗi CORS	Dùng <code>cors</code> middleware
API trả về undefined	Kiểm tra đúng format JSON + method
Server treo khi xử lý file lớn	Dùng Stream, không dùng <code>readFileSync</code>
Truy cập file sai đường dẫn	Dùng <code>path.join(__dirname, 'filename')</code>
Không phân biệt được method	Sử dụng middleware như <code>app.use(express.json())</code> để parse body

GỢI Ý CHO DỰ ÁN THỰC HÀNH

Dự án nhỏ	Kỹ thuật áp dụng
Todo List API	CRUD + Express + REST API
File Server đơn giản	Node.js http + fs
Website hiển thị HTML	HTTP server + <code>fs.readFile()</code>
Ứng dụng đăng nhập	API + POST + token (cơ bản)

EXERCISE

BÀI 1

TÓM TẮT DỰ ÁN

Mục tiêu dự án

- Tạo một **Web Server đơn giản bằng Node.js thuần (không dùng Express)**.
- Server sẽ trả về chuỗi **"Hello, World!"** mỗi khi có request từ trình duyệt.
- Hiểu cách hoạt động của module `http` trong Node.js.

Công nghệ sử dụng

Công cụ/Kỹ thuật	Mục đích
Node.js	Chạy JavaScript phía server
HTTP Module	Tạo Web Server
Nodemon (tuỳ chọn)	Tự động reload server khi thay đổi file
Windows Terminal (CMD/PowerShell)	Giao diện dòng lệnh

Chức năng

- Server lắng nghe tại `http://localhost:3000`.
- Mỗi khi có truy cập, server trả về **"Hello, World!"**.

Cấu trúc thư mục

pgsql

```
hello-server/  
├── index.js      # Tập tin chính chạy Node.js server  
└── package.json # Tập cấu hình Node.js
```

TÓM TẮT CÁC BƯỚC

1. Cài đặt Node.js trên Windows
2. Tạo thư mục dự án
3. Khởi tạo project với npm
4. Cài nodemon (tuỳ chọn)
5. Tạo file `index.js` và viết mã
6. Chạy server
7. Kiểm tra bằng trình duyệt

CHI TIẾT CÁC BƯỚC

BƯỚC 1: Cài đặt Node.js trên Windows

1. Truy cập: <https://nodejs.org>
2. Tải bản **LTS (Recommended For Most Users)** cho Windows.
3. Cài đặt theo mặc định, đảm bảo **"Add to PATH"** được chọn.

 Kiểm tra cài đặt:

Mở **Command Prompt (CMD)** và gõ:

```
bash
```

```
node -v
```

```
npm -v
```

BƯỚC 2: Tạo thư mục dự án

```
bash
```

```
mkdir hello-server
```

```
cd hello-server
```

BƯỚC 3: Khởi tạo project

```
bash
```

```
npm init -y
```

Tạo ra file `package.json` với thông tin mặc định.

BƯỚC 4: Cài nodemon (tùy chọn, tiện cho phát triển)

```
bash
```

```
npm install --save-dev nodemon
```

 Sau đó mở file `package.json`, cập nhật phần `"scripts"`:

```
json
```

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
}
```

BƯỚC 5: Tạo file `index.js`

Tạo file mới tên `index.js` bằng Notepad hoặc Visual Studio Code.

 Nội dung:

```
js
```

```
const http = require('http');
```

```
const hostname = 'localhost';
const port = 3000;

const server = http.createServer((req, res) => {
  console.log(`Request: ${req.method} ${req.url}`);

  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello, World!\n');
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

BƯỚC 6: Chạy server

◆ Cách 1: Chạy bằng Node.js (mỗi lần sửa phải chạy lại)

```
bash
npm start
```

◆ Cách 2: Chạy bằng nodemon (tự động reload sau khi sửa)

```
bash
npm run dev
```

BƯỚC 7: Mở trình duyệt và kiểm tra

- Mở **Chrome** hoặc **Edge**.
- Truy cập: `http://localhost:3000`
- Kết quả: Trang hiển thị `Hello, World!`

GIẢI THÍCH CÁC THÀNH PHẦN CHÍNH

Thành phần	Vai trò
<code>http.createServer()</code>	Tạo server
<code>req, res</code>	Request/Response từ client
<code>res.statusCode = 200</code>	Mã thành công
<code>res.setHeader(...)</code>	Thiết lập kiểu dữ liệu
<code>res.end(...)</code>	Kết thúc phản hồi và gửi nội dung

MẸO CHO WINDOWS

Mẹo	Cách thực hiện
Mở CMD tại thư mục	Shift + chuột phải → "Open PowerShell window here"
Tạo file nhanh	<code>notepad index.js</code>
Mở thư mục trong VS Code	<code>code .</code> (sau khi cài Visual Studio Code)

BÀI 2

Mục tiêu dự án

- Tạo một **RESTful API server** đơn giản bằng Node.js sử dụng Express.js.
- Thực hiện các thao tác CRUD (Create, Read, Update, Delete) với danh sách người dùng.
- Kiểm thử API bằng **Postman** (giao diện gửi request và xem phản hồi).

Công nghệ sử dụng

Công cụ/Kỹ thuật	Mô tả
Node.js	Nền tảng chạy JavaScript phía server
Express.js	Framework cho Node.js giúp xây dựng REST API dễ dàng
Postman	Công cụ kiểm thử và gửi HTTP request
JSON	Định dạng dữ liệu API trả về
Nodemon (<i>tuỳ chọn</i>)	Tự động reload server khi thay đổi code

Các chức năng cần có

- GET `/users` – Lấy danh sách người dùng
- GET `/users/:id` – Lấy thông tin người dùng theo ID
- POST `/users` – Tạo người dùng mới
- PUT `/users/:id` – Cập nhật thông tin người dùng
- DELETE `/users/:id` – Xóa người dùng

Tóm tắt các bước thực hiện

1. Tạo thư mục dự án và khởi tạo Node.js.
2. Cài đặt các gói cần thiết.
3. Tạo tệp chính `index.js`.
4. Viết code cho các route API.
5. Chạy server và kiểm thử với Postman.

CHI TIẾT CÁC BƯỚC THỰC HIỆN

BƯỚC 1: Tạo thư mục dự án và khởi tạo Node.js

bash

```
mkdir nodejs-api-users
cd nodejs-api-users
npm init -y
```


BƯỚC 2: Cài đặt các gói cần thiết

bash

```
npm install express
npm install nodemon --save-dev
```

 **Cập nhật** `package.json` để chạy bằng `nodemon`:

json

```
"scripts": {
  "start": "nodemon index.js"
}
```

BƯỚC 3: Tạo file `index.js`

bash

```
touch index.js
```

BƯỚC 4: Viết mã xử lý API trong `index.js`

js

```
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json()); // middleware để đọc JSON

// Dữ liệu mẫu
let users = [
  { id: 1, name: "Alice", email: "alice@example.com" },
  { id: 2, name: "Bob", email: "bob@example.com" }
];

// GET all users
app.get('/users', (req, res) => {
  res.json(users);
});

// GET user by ID
app.get('/users/:id', (req, res) => {
  const user = users.find(u => u.id == req.params.id);
  if (user) res.json(user);
  else res.status(404).json({ message: "User not found" });
});

// POST - Create new user
app.post('/users', (req, res) => {
  const newUser = {
    id: users.length + 1,
    ...req.body
  };
  users.push(newUser);
  res.status(201).json(newUser);
});

// PUT - Update user
app.put('/users/:id', (req, res) => {
```

```

const index = users.findIndex(u => u.id == req.params.id);
if (index !== -1) {
  users[index] = { id: Number(req.params.id), ...req.body };
  res.json(users[index]);
} else {
  res.status(404).json({ message: "User not found" });
}
});

// DELETE user
app.delete('/users/:id', (req, res) => {
  const index = users.findIndex(u => u.id == req.params.id);
  if (index !== -1) {
    const deleted = users.splice(index, 1);
    res.json(deleted[0]);
  } else {
    res.status(404).json({ message: "User not found" });
  }
});

// Khởi động server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});

```

BƯỚC 5: Chạy server

bash

npm start

BƯỚC 6: Kiểm thử API với Postman

► GET all users:

- **URL:** http://localhost:3000/users
- **Method:** GET

► GET user theo ID:

- **URL:** http://localhost:3000/users/1

► POST tạo user mới:

- **URL:** http://localhost:3000/users
- **Method:** POST
- **Body (JSON):**

json

```

{
  "name": "Charlie",
  "email": "charlie@example.com"
}

```

► PUT cập nhật user:

- **URL:** `http://localhost:3000/users/2`
- **Method:** PUT
- **Body (JSON):**

json

```
{
  "name": "Bob Updated",
  "email": "bobnew@example.com"
}
```

► DELETE user:

- **URL:** `http://localhost:3000/users/2`
- **Method:** DELETE

✓ KẾT LUẬN

Bạn đã:

- Tạo một REST API hoàn chỉnh bằng Node.js và Express
- Hiểu các phương thức HTTP: GET, POST, PUT, DELETE
- Thực hành kiểm thử bằng Postman

BÀI 3

Mục tiêu

- Tạo một **HTTP Server bằng Node.js** có khả năng trả về các file `.html` tương ứng với đường dẫn URL.
- Thực hành đọc file bằng `fs` và xử lý đường dẫn bằng `path`.

Công nghệ sử dụng

Công nghệ	Mục đích
Node.js	Nền tảng JavaScript phía server
HTTP Module	Tạo Web Server
fs module	Đọc file hệ thống
path module	Xử lý đường dẫn file
Nodemon (tuỳ chọn)	Tự động reload server khi thay đổi

Chức năng

- Trả về nội dung file:
 - `/` → `index.html`
 - `/about` → `about.html`
 - `/contact` → `contact.html`
- Nếu file không tồn tại → trả về `404.html`.

Cấu trúc thư mục

css

```
file-server/  
├── index.js           # Tập chính Node.js  
├── package.json      # Tập cấu hình npm  
└── public/           # Thư mục chứa HTML  
    ├── index.html  
    ├── about.html  
    ├── contact.html  
    └── 404.html
```

TÓM TẮT CÁC BƯỚC THỰC HIỆN

1. Tạo thư mục và khởi tạo dự án Node.js
2. Tạo thư mục `public/` và thêm các file HTML
3. Viết mã Node.js để xử lý URL và đọc file
4. Chạy server và kiểm tra kết quả

CHI TIẾT CÁC BƯỚC

BƯỚC 1: Tạo thư mục và khởi tạo dự án

bash

```
mkdir file-server  
cd file-server  
npm init -y  
npm install --save-dev nodemon
```

Cập nhật `package.json`:

json

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js"  
}
```

BƯỚC 2: Tạo thư mục `public/` và thêm file HTML

Tạo thư mục và các file:

bash

```
mkdir public
```

File: `public/index.html`

html

```
<h1>Trang chủ</h1>
<p>Chào mừng bạn đến với website!</p>
```

File: public/about.html

html

```
<h1>Giới thiệu</h1>
<p>Đây là trang giới thiệu.</p>
```

File: public/contact.html

html

```
<h1>Liên hệ</h1>
<p>Thông tin liên hệ ở đây.</p>
```

File: public/404.html

html

```
<h1>404 - Không tìm thấy</h1>
<p>Trang bạn yêu cầu không tồn tại.</p>
```

 BƯỚC 3: Viết mã trong index.js
js

```
const http = require('http');
const fs = require('fs');
const path = require('path');

const hostname = 'localhost';
const port = 3000;

const server = http.createServer((req, res) => {
  console.log(`Request: ${req.method} ${req.url}`);

  let filePath = './public';
  let fileName = req.url === '/' ? '/index.html' : req.url + '.html';
  filePath = path.resolve(filePath + fileName);

  const ext = path.extname(filePath);

  if (ext !== '.html') {
    res.statusCode = 403;
    res.setHeader('Content-Type', 'text/plain');
    res.end('403 - Only HTML files are allowed');
    return;
  }

  fs.access(filePath, fs.constants.F_OK, (err) => {
    if (err) {
      const errorPath = path.resolve('./public/404.html');
      fs.readFile(errorPath, (err, data) => {
        res.statusCode = 404;
        res.setHeader('Content-Type', 'text/html');
        res.end(data);
      });
    } else {

```

```

    fs.readFile(filePath, (err, data) => {
      res.statusCode = 200;
      res.setHeader('Content-Type', 'text/html');
      res.end(data);
    });
  }
});
});
});

server.listen(port, hostname, () => {
  console.log(`Server is running at http://${hostname}:${port}/`);
});

```

BƯỚC 4: Chạy server

bash

npm run dev

BƯỚC 5: Kiểm tra trên trình duyệt

URL	Nội dung trả về
http://localhost:3000/	index.html
http://localhost:3000/about	about.html
http://localhost:3000/contact	contact.html
http://localhost:3000/abc	404.html

GIẢI THÍCH THÊM

Thành phần	Vai trò
path.resolve()	Tạo đường dẫn tuyệt đối
fs.access()	Kiểm tra file có tồn tại không
fs.readFile()	Đọc nội dung file HTML
path.extname()	Kiểm tra phần mở rộng của file

MỞ RỘNG GỢI Ý

- Thêm hỗ trợ cho CSS, hình ảnh (.css, .jpg) bằng cách mở rộng MIME.
- Ghi log truy cập vào file log.txt.
- Phân tích req.headers để xem IP, User-Agent.